

PCL-PUF: Physically Coupled Layered PUF for Machine Learning Resilience

Mohamed Alsharkawy*, Hassan Nassar*, Jeferson Gonzalez-Gomez[†], Kuan-Hsun Chen[‡], and Jörg Henkel*

*Chair for Embedded Systems (CES), Karlsruhe Institute of Technology (KIT), Germany

[†]Costa Rica Institute of Technology (TEC), Costa Rica

[‡]University of Twente (UT), Netherlands

Emails: mohamed.alsharkawy@kit.edu, hassan.nassar@kit.edu, jgonzalez@itcr.ac.cr, k.h.chen@utwente.nl, henkel@kit.edu

Abstract—Physical Unclonable Functions (PUFs) are widely used as lightweight security primitives for resource-constrained devices, particularly for device authentication and key generation. Although PUFs are unclonable from one chip to another, they are vulnerable to Machine Learning (ML) modeling attacks. State-of-the-art designs improve resilience against specific attacks but remain vulnerable to other types of attacks. We propose the Physically Coupled Layered PUF (PCL-PUF), a strong PUF that physically couples silicon delay-based and non-volatile memory-based PUF primitives. This physical coupling induces a dynamically varying challenge-response behavior and prevents independent learning of internal structures, significantly increasing modeling complexity. We evaluate PCL-PUF against logistic regression, reliability, splitting, and neural network attacks. We show that logistic regression, reliability, and splitting attacks fail against PCL-PUF. Moreover, under neural network attacks with up to 10^7 training samples, PCL-PUF remains unbroken, with modeling accuracy limited to 50%, whereas state-of-the-art designs of similar hardware size can be modeled with more than 98%. Compared to the state-of-the-art PUF designs, PCL-PUF remains secure against all major classes of modeling attacks.

Index Terms—Physical Unclonable Function, Machine Learning Resilience, Non-volatile Memory

I. INTRODUCTION

Physical Unclonable Functions (PUFs) are a widely used hardware security primitive that exploits manufacturing variations in integrated circuits to generate unique, device-specific fingerprints. A PUF operates as a one-way function that takes a *challenge* as input and produces a corresponding *response*, together forming a Challenge-Response Pair (CRP) [1], [2]. The uniqueness of these fingerprints combined with the lightweight nature of PUF circuits makes them attractive for various security applications, such as authentication and key generation in resource-constrained IoT devices [3], [4].

PUFs are broadly categorized as *weak* or *strong* depending on the number of CRPs they offer [5], [6]. Weak PUFs provide a limited number of CRPs, making them vulnerable to brute-force attacks where an adversary can record all responses and impersonate the PUF using a simple lookup table [4]. Strong PUFs, by contrast, offer an exponential number of CRPs, making brute-force attacks infeasible [4].

Although strong PUFs offer an exponential CRP space, they are highly vulnerable to Machine Learning (ML) modeling attacks, in which an adversary collects a subset of CRPs and

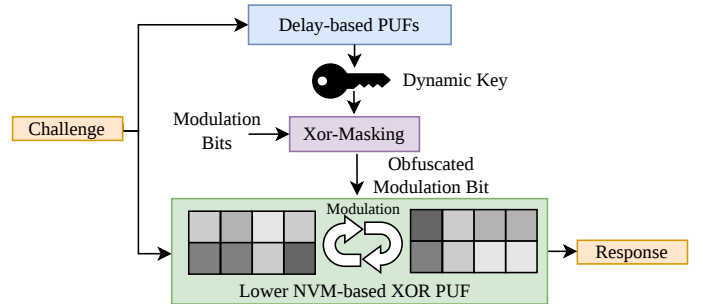


Fig. 1: Overview of the proposed Physically Coupled Layered PUF (PCL-PUF). Upper-layer delay-based PUFs generate dynamic responses that iteratively XOR mask the MLC modulation bits controlling the lower-layer NVM-based XOR PUF. This establishes physical coupling between the two layers, which induces a dynamically varying challenge-response behavior and prevents decoupling or independent learning.

trains a model that accurately predicts responses to unseen challenges [4], [7], [8]. Numerous countermeasures have been proposed to harden strong PUFs against such attacks, yet each leaves at least one attack vector open.

XOR PUFs [9] increase the nonlinearity of the challenge-response behavior by XORing multiple Arbiter PUF (APUF) outputs, which improves resilience against Neural Network (NN) attacks [10] but remains vulnerable to Logistic Regression (LR) [7] and reliability attacks [11]. The Interpose PUF (iPUF) [12] advances XOR PUF design by using two XOR-APUF, where the output of the upper PUF is inserted into the challenge of the lower PUF to introduce stronger nonlinear behavior. This mitigates LR and reliability attacks but remains vulnerable to splitting attacks [8], which isolate and iteratively model each PUF independently.

Unlike structurally composed designs, NVM-based PUFs mitigate modeling attacks by exploiting the inherent nonlinearity of Multi-Level Cell (MLC) resistance characteristics [13]–[15]. By programming NVM cells to different resistance states, they influence the delay behavior of the underlying PUF and create variable responses that improve resilience to modeling attacks [13]–[15]. Despite these claims, we show in Section V-B4 that NVM-based PUFs can still be broken by NNs given sufficient training data.

To address the limitations of current state-of-the-art designs, we propose the Physically Coupled Layered PUF (PCL-PUF), a novel ML-resilient PUF that integrates an upper layer of delay-based PUFs with a lower layer of NVM-based XOR PUF as shown in Figure 1. The upper-layer responses obfuscate the MLC modulation bits of the lower layer through an iterative XOR masking scheme, physically coupling the two layers. This coupling induces dynamically varying challenge-response behavior, and prevents the two layers from being decoupled or learned independently, significantly increasing modeling complexity.

We evaluate PCL-PUF against four categories of state-of-the-art modeling attacks: LR [7], reliability [11], splitting [8], and NN attacks [10]. For the first two, we provide mathematical arguments that they are infeasible against our design and confirm this through experimental evaluation. We further show that PCL-PUF achieves significantly higher resilience against splitting attacks compared to the iPUF [12], the only prior design resilient to both LR and reliability attacks. Finally, under NN attacks with up to 10^7 CRPs, all PCL-PUF configurations remain unbroken: for the same configuration (i.e., the same number of PUF instances), state-of-the-art designs reach more than 98% modeling accuracy, whereas PCL-PUF remains limited to 50%. Together, these results demonstrate that PCL-PUF simultaneously mitigates all four attack categories, a property not achieved by any prior design. In summary, the major contributions of this work are:

- We introduce PCL-PUF, a PUF architecture that physically couples delay-based and NVM-based PUFs. This physical coupling creates a dynamically varying challenge-response behavior and prevents independent modeling of internal structures significantly increasing modeling complexity.
- We propose physical cross-layer coupling as a promising design principle for ML-resilient PUFs and show how PCL-PUF overcomes the limitations of prior approaches (e.g., XOR PUF and iPUF).
- We show through comprehensive evaluation that PCL-PUF is resilient against four major attack classes: LR [7], reliability [11], splitting [8], and NN [10] attacks outperforming state-of-the-art PUF designs.

II. BACKGROUND

A. Physical Unclonable Functions (PUFs)

PUFs exploit variations in the device fabrication process to produce behavior that is random across devices but reproducible for the same device [16]. These responses are inherently unique and cannot be cloned even by the PUF designer.

1) *APUFs*: APUF [1] is one of the most widely used PUF designs, and many PUF-based protocols are built on APUFs or similar delay-based architectures [4]. Its architecture consists of a sequence of multiplexers (MUXs) stages, where each bit of the input challenge vector controls a pair of MUXs. The operating principle relies on a race signal propagating simultaneously through two nominally identical delay paths, with the challenge bits determining the exact routing through each stage of the MUX chain [17].

Due to small variations introduced during manufacturing, each stage has slightly different propagation delays which causes subtle differences in the arrival times of the signals at the end of the paths. This makes the final timing outcome sensitive to both the challenge and the device's physical characteristics. Finally, an arbiter element determines which of the two signals arrives first and outputs a single-bit response [4], [17].

2) *Quality Metrics*: The performance of a PUF is commonly evaluated using three key metrics: uniqueness, uniformity, and reliability, which characterize response distinctiveness, statistical balance, and stability, respectively [18].

Uniqueness measures how distinct responses are across different PUF instances under the same challenge. Ideally, it should be 50%, indicating uncorrelated responses with equal bit-flip probability. It is defined as

$$\text{Uniqueness} = \frac{2}{N(N-1)} \sum_{x=1}^{N-1} \sum_{y=x+1}^N \frac{\text{HD}(R_x, R_y)}{n}, \quad (1)$$

where N is the number of devices, R_x and R_y are the responses of devices x and y , n is the response length, and HD denotes the Hamming distance.

Uniformity measures the balance between '1's and '0's in a single PUF response. An ideal value of 50% indicates an unbiased response distribution. It is defined as

$$\text{Uniformity} = \frac{\sum_{i=1}^n R_i}{n}, \quad (2)$$

where n is the response length and R_i is the i -th response bit.

Reliability measures the stability of PUF responses under varying environmental conditions by comparing noisy responses to a reference response. Ideally, reliability should approach 100%. It is defined as

$$\text{Reliability} = \frac{1}{m} \sum_{x=1}^m \left(1 - \frac{\text{HD}(R, R'_x)}{n} \right), \quad (3)$$

where m is the number of tests, R is the reference response, R'_x is the response under test conditions, and n is the response length.

B. ML Modeling Attacks

ML modeling attacks are a primary threat to PUF security. In these attacks, an adversary collects a subset of CRPs from the target PUF and uses them to train a model that learns the underlying challenge-response behavior, enabling accurate prediction of responses to unseen challenges [4], [7], [8].

ML modeling attacks, such as LR [7] and support vector machines (SVMs) [19], exploit the known mathematical structure of the target PUF. For delay-based PUFs such as APUFs and their variants, the responses are governed by a linearly additive delay structure [4]. The adversary transforms the challenge bits into a feature vector that encodes these linear delay dependencies, which is then used together with the PUF responses as labels to train a classical ML model that can efficiently approximate the PUF behavior.

NNs [10] take a fundamentally different approach by operating as black box optimizers that do not require any knowledge

of the underlying mathematical model. Instead, they learn the challenge-response behavior directly from raw challenge bits and observed responses. Although this makes them more general and applicable to a wider range of PUF designs, they typically require a larger number of CRPs to achieve high modeling accuracy compared to classical approaches [12].

C. Non-volatile memory

Non-Volatile Memory (NVM) has emerged as an attractive alternative for conventional volatile memories such as DRAM and SRAM [20], [21]. One of the key features of NVM is its support for MLC operation, which allows a single memory cell to represent multiple logical states through distinct resistance levels. Various memory technologies are used to produce NVMs, including Phase Change Memory (PCM), Resistive RAM (ReRAM), Spin-Orbit Torque RAM (SOT-RAM), and Spin-Transfer Torque RAM (STT-RAM) [22]. Although these technologies differ in their physical implementation, they all exhibit the ability to reliably transition between multiple resistance states [23], [24].

The availability of multiple resistance levels in NVM devices, together with intrinsic cell-to-cell resistance variations even under identical programming conditions [25], has enabled their use as PUFs [14], [26], [27]. These PUFs exploit such resistance variations as a source of entropy by mapping them to timing or delay differences in the PUF circuitry. Furthermore, programming NVM cells to multiple levels enables multiple variable delay per cell, improving resilience against modeling attacks [14], [15], [26].

III. RELATED WORK

Numerous countermeasures have been proposed to develop ML-resilient PUFs. Nevertheless, even these enhanced designs remain susceptible to advanced modeling attacks. In the following, we review these countermeasures and the corresponding attacks specifically designed to target them.

Output obfuscation techniques, such as the XOR-PUF [9], combine the outputs of multiple APUF instances using XOR operations, introducing nonlinear challenge-response behavior that significantly increases the complexity of modeling attacks. However, [7], [28] demonstrated that XOR PUFs remain vulnerable to LR attacks. Specifically, [28] showed that 128-bit l -XOR APUFs with up to $l = 7$ instances can be modeled via LR, which jointly optimizes a weight vector per APUF instance over the collected CRPs. For a larger number of XORed instances, however, LR fails due to the sub-exponential growth in the amount of training data required as l increases [7], [28].

The security of XOR PUFs with $l > 7$ was broken by short-term reliability attacks [11], which reduce the attack complexity to linear in l . The attack exploits the observation that when the delay difference $|\Delta|$ between the two competing paths of an APUF falls below a noise threshold ϵ , the response becomes unreliable under intrinsic circuit noise [11], [29]. By collecting E repeated measurements per challenge, the attacker

estimates the short-term reliability of the PUF and constructs the following noise information signal:

$$\mathcal{I}(i) = \left| \frac{E}{2} - \sum_{e=1}^E r_i^{(e)} \right|, \quad (4)$$

where $r_i^{(e)} \in \{0, 1\}$ is a binary indicator of the e -th repeated evaluation of challenge $\mathbf{c}_i$, with $r_i^{(e)} = 1$ indicating that the response is noisy and $r_i^{(e)} = 0$ indicating a stable response. A value of $\mathcal{I}(i) = E/2$ corresponds to a fully reliable challenge, whereas $\mathcal{I}(i) \approx 0$ indicates an unreliable challenge. The attacker then uses Covariance Matrix Adaptation Evolution Strategy (CMA-ES) to find model weights $\mathbf{w}^*$ that maximize the correlation between the model-predicted margin and the observed reliability:

$$\mathbf{w}^* = \arg \max_{\mathbf{w}} \text{corr}(|\Phi \mathbf{w}|, \mathcal{I}), \quad (5)$$

recovering the true internal delay parameters when this correlation is achievable.

iPUF [12] was proposed as a strong PUF design resilient against both LR and reliability attacks. It consists of two layers of XOR APUFs, denoted by (K_u, K_d) , where K_u and K_d represent the number of APUFs in the upper and lower layers, respectively. The upper layer processes an n -bit challenge and produces a single-bit response, which is interposed into the middle of the lower-layer challenge, extending it to $n + 1$ bits as shown in Figure 2. This structure mitigates reliability attacks because successful reliability attacks on layered PUFs require all APUF instances to contribute comparably to the final output and exhibit similar noise rates [12]. In iPUF, however, the upper and lower layers contribute unequally: when both layers share the same noise rate β , the effective contribution of the upper layer is only $\beta/2$ compared to β for the lower layer, violating the comparable-influence condition required by the attack. The layered structure also mitigates LR modeling attacks. The authors argue that the design is only vulnerable to NN attacks, since such attacks do not require knowledge of the underlying mathematical model; however, they require a much larger number of CRPs.

Despite the resilience of the iPUF against classical attacks and the reliability CMA-ES attack, the authors in [8] proposed a splitting attack that disables the effect of the layered structure of the iPUF. It eliminates the dependency between the upper and lower PUF stages by first modeling the lower layer independently. Although the input to the lower layer depends on the interposed bit produced by the upper layer, which is unknown to the attacker, the attacker circumvents this dependency by randomly guessing the interpose bit. The attacker constructs a custom dataset for the lower layer in which uniformly random bits are inserted in place of the unknown interpose bit, and uses it to model the lower layer. The upper-layer response is then recovered using the lower-layer model, specifically by filtering out all challenges for which the upper-layer response bit does not influence the final output. For the remaining challenges, the lower-layer model is evaluated under both possible interpose bit values, and the value that produces the correct final response

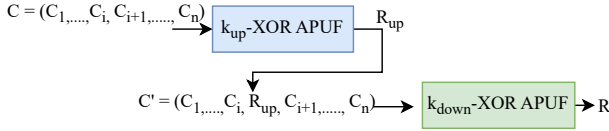


Fig. 2: Abstract design of Interpose PUF (iPUF)

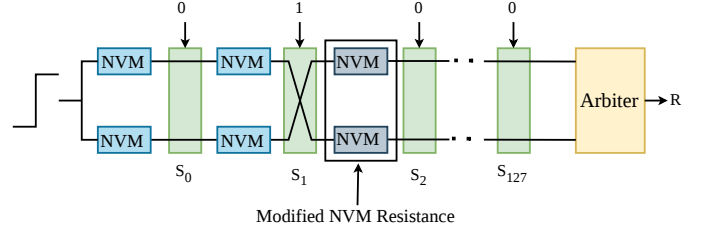


Fig. 3: Abstract design of REAP-NVM PUF

is assigned as the upper-layer response. This dataset is then used to model the upper layer. After modeling both layers, an iterative divide-and-conquer attack models the complete iPUF. This splitting attack successfully manage to model iPUF with configurations up to $(K_u = 8, K_d = 8)$ [8], where K_u and K_d is the number of APUF in the upper and lower layer respectively.

Structural obfuscation has also been explored independently. PLPUF [30] proposes a reconfigurable PUF that changes its challenge–response behavior at runtime by adjusting its activation duration, while CaPUF [31] extends this by employing an array of PLPUFs where the output of one influences the activation duration of another. However, recent work [32] has shown that PLPUF uniqueness is highly dependent on activation duration, enabling an attacker with a reference PUF to model it accurately by exploiting response correlations at specific activation durations. This attack can also be extended to CaPUF.

NVM-based PUFs have gained significant attention due to their inherent MLC characteristics, which enhance resilience against ML modeling attacks [13]–[15]. ANV-PUF [14] utilizes the MLC property of NVM devices instead of relying on delay propagation through switching elements as in conventional APUFs. It operates by specifying a target resistance for each NVM cell and recording the number of programming pulses required to reach that resistance. This pulse count is mapped to two adders based on challenge bits, and a comparator produces the final response bit based on which adder sum is larger. Due to the inherently nonlinear and device-specific resistance evolution of NVM cells, the resulting challenge–response behavior exhibits strong nonlinearity.

Despite their strong resilience to ML attacks, NVM-based PUFs suffer from limited endurance, as frequent write operations accelerate wear and degrade the underlying NVM cells. REAP-NVM PUF [13] addresses this issue by updating only a single pair of NVM cells per query, instead of programming all cells during each evaluation, thereby improving endurance by up to $62\times$ while maintaining comparable ML resilience. REAP-NVM PUF is built upon an APUF, as illustrated in Figure 3, where the resistance states of NVM cells serve as delay sources. It exploits the MLC capability of NVM devices to modify the delay contribution of a selected pair of cells, effectively creating a different PUF instance for each query. To support this mechanism, REAP-NVM employs a challenge structure consisting of switch-network bits of length n and physical modulation bits of length $m = \log_2 n + \log_2 l$, where n denotes the number of stages and l denotes the number of resistance levels supported by the underlying NVM device. Specifically, $\log_2 n$ bits of physical modulation bits select the target pair

of NVM cells, while the remaining $\log_2 l$ bits determine the encoded resistance state, directly modifying the resulting delay and increasing modeling difficulty. Despite these claims, NVM-based PUFs have not been evaluated under large-scale attacks: REAP-NVM PUF [13] and MAPUF [15] were tested with fewer than 10^5 CRPs, leaving their resilience under large-scale attack scenarios unexplored.

Previous strong PUF designs mitigate some modeling attacks but remain vulnerable to others. XOR PUFs [9] are vulnerable to both LR and reliability attacks [7], [11]. The iPUF [12] mitigates these attacks through a layered structure but remains vulnerable to splitting attacks [8]. Although NVM-based PUFs are often considered ML resilient due to their physical properties, we show in Section V-B4 that they can also be broken by NN attacks given sufficient training data.

IV. PHYSICALLY COUPLED LAYERED PUF (PCL-PUF)

To address the limitations of current state-of-the-art designs, we propose a novel Physically Coupled Layered PUF (PCL-PUF) that achieves greater modeling complexity and improved resilience compared to previous approaches. We analyze the state-of-the-art attacks to build our PUF to be resilient against them. Moreover, we mathematically analyze its modeling complexity to ensure this resilience is achieved.

A. Threat Model

We consider an attacker who can collect CRPs to launch modeling attack. The attacker can do this either by eavesdropping on the communication between the verifier and the PUF, by impersonating the verifier, or by temporarily gaining physical access to the device (e.g. scammer devices on ATMs). The attacker’s goal is to build a model that predicts the PUF response for any unseen challenge with high accuracy, i.e., more than 90%. We consider four attack categories, each targeting a different structural vulnerability.

1) *Logistic Regression (LR) attacks*: LR is considered one of the most effective modeling attacks against delay-based PUF designs [12], as the attacker uses knowledge of the PUF architecture to build a mathematical model of the challenge–response relationship and trains it on collected CRPs to learn the PUF behavior. This attack is highly effective against XOR PUFs [7].

2) *Reliability attacks*: The attacker exploits repeated noisy measurements to extract the internal delay margins of the PUF [11]. This attack is highly effective against XOR PUFs.

3) *Splitting Attacks*: The attacker targets PUF designs with a layered architecture by modeling one layer first and using it to reconstruct the other, applying a divide-and-conquer strategy to model the complete design. This attack is highly effective against the iPUF [8].

4) *Neural Network (NN) attacks*: The attacker uses NN to model a PUF as a black box with no structural assumptions or mathematical model. This attack is applicable to all PUF designs but requires a large number of CRPs to converge, making it a benchmark for PUF security.

B. Our Novel PUF Design

Our goal is to design a PUF that is structurally resilient against the first three attack classes defined in the threat model, while maximizing the number of CRPs required for NN attacks to achieve meaningful prediction accuracy beyond practical limits. In particular, the required number of CRPs should exceed what can realistically be collected within the endurance limits of NVM. While reported NVM endurance varies widely depending on device optimization, practical MLC NVM typically operates in a mid-range between performance-optimized and endurance-optimized designs. We therefore consider 10^7 CRPs a realistic upper bound for attack evaluation [33].

To achieve our design goals, we propose PCL-PUF, which employs a layered architecture in which the upper layer obfuscates the physical behavior of the lower layer, as illustrated in Figure 4. The design integrates an upper layer consisting of K_u delay-based PUFs and a lower layer consisting of K_d NVM-based PUFs. The upper-layer delay-based PUFs generate responses that directly control the state of MLCs in the lower layer. Each MLC is programmed to a specific resistance level, which determines the corresponding delay characteristics. By modulating the resistance levels through the upper-layer outputs, the design dynamically alters the delay behavior of the lower-layer PUF, effectively coupling the two layers at the physical level and preventing independent learning of their internal structures as we show in Section V-B3.

This physical coupling causes the lower-layer PUFs to exhibit dynamically varying challenge-response behavior, as different upper-layer responses generated by different input challenges create different delay configurations across the challenge space, substantially increasing modeling complexity. While each layer alone does not guarantee strong resilience against modeling attacks, combining the two layers in PCL-PUF provides significantly enhanced ML resilience, as shown in Section V-B.

1) *Lower layer*: Our lower layer consists of multiple XORed NVM-based PUFs, implemented using REAP-NVM PUFs [13]. REAP-NVM PUF can modulate the delay of the lower layer through physical modulation bits, and has an improved endurance compared to other NVM-based PUF designs, making it an ideal choice for our lower-layer PUF. As discussed in Section III, REAP-NVM PUF use n bits for the switch network and $m = \log_2 n + \log_2 l$ physical modulation bits, where $\log_2 n$ bits select one cell pair out of n cell pairs, and $\log_2 l$ bits encode the target resistance level of the selected cell pair.

2) *Upper Layer*: To obfuscate the physical behavior of the lower-layer NVM-based PUFs so that their state cannot be easily predicted by machine-learning models, our PCL-PUF employs an upper layer consisting of multiple APUFs. The outputs of these APUFs are used to select the target NVM cell pair and determine its encoding value for each query via physical modulation bits. A straightforward approach is to have the upper layer directly generate a response of m bits, corresponding to the total number of physical modulation bits. Although this solution provides strong security, it introduces area overhead, as it would require m APUFs in the upper layer.

3) *XOR Masking*: To optimize the area while offering strong security, we propose using the upper-layer APUFs outputs to obfuscate the physical modulation bits rather than directly generating them. The upper-layer APUFs outputs are used to iteratively XOR mask the m physical modulation bits of the lower-layer NVM-based PUFs, such that each modulation bit is XORed exactly once.

Figure 5 illustrates this process for an upper-layer response of three bits applied to ten physical modulation bits. In each iteration, the three upper-layer bits are XORed with three consecutive modulation bits. In the final iteration, the remaining single modulation bit is XORed with the first bit of the upper-layer response.

This ensures that all physical modulation bits depend directly on the upper-layer response, effectively hiding the obfuscated physical modulations values from an attacker. We further show in Section V-A that this technique preserves the inherent randomness of the physical modulation bits and does not introduce bias.

C. PCL-PUF Operation

PCL-PUF supports different (K_u, K_d) configurations; however, all configurations follow the same operating principle. It is queried with a challenge consisting of n switch bits and m physical modulation bits. The n switch bits are applied to the upper layer, which generates K_u response bits that are not exposed externally. These response bits are used to iteratively XOR mask the physical modulation bits, producing an obfuscated physical modulation vector. Finally, the K_d lower-layer REAP-NVM PUFs evaluate the challenge using the original n switch bits together with the obfuscated modulation bits, and the outputs of the K_d instances are XORed to produce the final response of PCL-PUF.

D. Modeling Complexity of PCL-PUF

We provide a mathematical analysis showing that LR attacks are infeasible and that reliability attacks are extremely difficult against PCL-PUF. First, we derive the mathematical model of the proposed design to clarify its operating principle and then analyze its resilience to both attack types mathematically. These findings are further confirmed by the experimental evaluation presented in Section V-B.

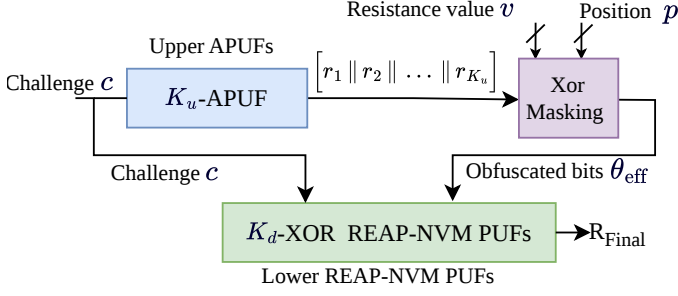


Fig. 4: Proposed PCL-PUF architecture, consisting of an upper layer of multiple APUFs and a lower layer of multiple REAP-NVM PUFs. The upper-layer responses obfuscate the physical modulation bits of the lower layer through iterative XOR masking, physically coupling the two layers.

1) *Mathematical Model:* The upper layer consists of K_u APUF instances, each producing a response based on the delay difference between two racing paths. Following the linear additive delay model [4], [7], the delay difference of k -th APUF instance is expressed as

$$\Delta_{u,k} = \langle \mathbf{w}_k^u, \Psi \rangle = \sum_{i=0}^n w_k^u[i] \Psi[i], \quad (6)$$

where n is the number of challenge bits, $\mathbf{w}_k^u$ is the weight vector encoding the process-variation of the stage delays for the k -th APUF in the upper layer, and Ψ is the parity (feature) vector derived from the challenge c as

$$\Psi[n] = 1, \quad \Psi[i] = \prod_{j=i}^{n-1} c[j], \quad i = 0, \dots, n-1. \quad (7)$$

The parity vector Ψ encodes the cumulative product of the challenge bits from position i onward, capturing the fact that flipping $c[i]$ affects signal routing through all subsequent stages. The final entry $\Psi[n] = 1$ serves as a bias term that accounts for the fixed delay offset [4], [7]. The response of the k -th APUF instance is

$$r_k = \delta(\Delta_{u,k}) \in \{-1, +1\}, \quad (8)$$

where $\delta(\cdot)$ is the step function, i.e., $\delta(x) = 1$ if $x > 0$ and $\delta(x) = 0$ otherwise. The function δ has derivative 0 everywhere except for $x = 0$ (where the derivative is ∞), and since $\Delta_{u,k}$ is never exactly equal to zero in practice, we may assume that the derivative of δ in $\Delta_{u,k}$ is always zero.

Since the upper layer contains K_u independent APUFs, its output is the concatenation of the individual response bits:

$$f_u(c) = [r_1 \parallel r_2 \parallel \dots \parallel r_{K_u}], \quad (9)$$

The lower layer consists of K_d REAP-NVM PUF instances. REAP-NVM PUFs extend the conventional APUF by utilizing NVM cell resistances as programmable delay sources. Let $\theta = (p, v)$ denote the physical modulation parameters, where $p \in \{0, \dots, n-1\}$ is the selected stage and $v \in \{0, \dots, l-1\}$ is the selected resistance level, where $v = 0$ corresponds to

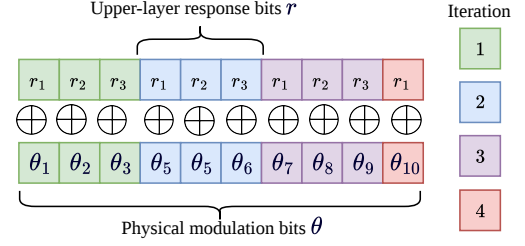


Fig. 5: Illustration of the XOR masking process, where three upper-layer response bits are applied to ten physical modulation bits. Each iteration (highlighted in a distinct color) XORs the upper-layer bits with three consecutive modulation bits. In the final iteration, the remaining modulation bit is XORed with the first upper-layer response bit.

the base resistance. The weight vector $\mathbf{w}_k^d(\theta)$ of k -th REAP-NVM PUF can be expressed as a base weight vector $\mathbf{w}_{k,\text{base}}^d$ with a single-stage modification at position p determined by the selected resistance level v :

$$\mathbf{w}_k^d(\theta) = \begin{cases} w_{k,\text{base}}^d[i] & \text{if } i \neq p, \\ w_k^{d,(v)}[p] & \text{if } i = p, \end{cases} \quad (10)$$

where $w_k^{d,(v)}[p]$ is the delay contribution of the NVM level v at stage p . The delay difference of the k -th REAP-NVM PUF instance is:

$$\Delta_{d,k}^{\theta} = \langle \mathbf{w}_k^d(\theta), \Psi \rangle = \underbrace{\langle \mathbf{w}_{k,\text{base}}^d, \Psi \rangle}_{\text{base delay}} + \underbrace{(w_k^{d,(v)}[p] - w_{k,\text{base}}^d[p]) \Psi[p]}_{\text{NVM correction}}, \quad (11)$$

where $\Psi[p]$ denotes the p -th component of the parity vector Ψ . The response of all K_d XORed instances according to [7] is:

$$\Delta_d^{\theta} = \prod_{k=1}^{K_d} \langle \mathbf{w}_k^d(\theta), \Psi \rangle, \quad R_L = \delta(\Delta_d^{\theta}). \quad (12)$$

Instead of directly applying θ to the lower-layer instances, PCL-PUF obfuscates the lower-layer configuration using both θ and the upper-layer output $f_U(c)$:

$$\theta_{\text{eff}} = g(\theta, f_U(c)), \quad (13)$$

where $g(\cdot)$ represents the iterative XOR masking shown in Fig. 5 that combines θ with the upper-layer responses. Substituting θ_{eff} into the lower-layer model, the final PCL-PUF response is

$$R_{\text{Final}} = \delta \left(\prod_{k=1}^{K_d} \langle \mathbf{w}_k^d(g(\theta, f_U(c))), \Psi \rangle \right). \quad (14)$$

2) *Infeasibility of LR:* We show that LR **cannot** optimize the PCL-PUF model and that at best it can learn a linearly approximated model of our PUF, where the attacker assumes that the upper layer has a constant output across all challenges, meaning that the attacker treats θ_{eff} as θ which is independent of the challenge. To launch an LR modeling attack, The attacker approximates the discrete final output R_{Final} with the continuous

sigmoid $\sigma(x) = (1 + \exp(-x))^{-1}$ and minimizes the cross-entropy loss over a training set of N CRPs:

$$l = -\frac{1}{N} \sum_{i=1}^N \ln \left(\sigma(\Delta_d^{\theta_{\text{eff}}, i})^{R_i} \left(1 - \sigma(\Delta_d^{\theta_{\text{eff}}, i}) \right)^{1-R_i} \right), \quad (15)$$

where $\Delta_d^{\theta_{\text{eff}}, i} = \prod_{k=1}^{K_d} \langle \mathbf{w}_k^d(\theta_{\text{eff}}), \Psi_i \rangle$ is the lower-layer delay margin for the i -th challenge under θ_{eff} , and R_i is the observed PCL-PUF response. The attacker seeks to jointly optimize the full weight set $\mathbf{w} = (\mathbf{w}^u, \mathbf{w}^d)$ by gradient descent in an attempt to recover the full PCL-PUF model, where $\mathbf{w}^u$ represents the weight sets of the K_u upper-layer APUFs, and $\mathbf{w}^d$ represents the weight sets of the K_d lower-layer's REAP-NVM PUFs. For the proof, we focus on the partial derivative with respect to the hidden upper-layer weights $\mathbf{w}^u[m]$, where $m = 0, \dots, n$. Applying the chain rule to the k -th APUF:

$$\frac{\partial \ell}{\partial \mathbf{w}_k^u[m]} = \frac{\partial \ell}{\partial \Delta_d^{\theta_{\text{eff}}}} \cdot \frac{\partial \Delta_d^{\theta_{\text{eff}}}}{\partial \mathbf{w}_k^d} \cdot \frac{\partial \mathbf{w}_k^d}{\partial \theta_{\text{eff}}} \cdot \frac{\partial \theta_{\text{eff}}}{\partial r_k} \cdot \frac{\partial r_k}{\partial \mathbf{w}_k^u[m]}. \quad (16)$$

Following [12], the attacker models the discrete upper APUF with an additional differentiable error term $e(\Delta_{u,k})$ where $e(\cdot)$ is a differentiable function chosen by the attacker:

$$\hat{r}_k = \delta(\Delta_{u,k}) + e(\Delta_{u,k}), \quad (17)$$

since $\delta(\cdot)$ has derivative zero everywhere the derivative of the loss reduces to:

$$\begin{aligned} \frac{\partial \ell}{\partial \mathbf{w}_k^u[m]} &= \frac{\partial \ell}{\partial \Delta_d^{\theta_{\text{eff}}}} \cdot \frac{\partial \Delta_d^{\theta_{\text{eff}}}}{\partial \mathbf{w}_k^d} \cdot \frac{\partial \mathbf{w}_k^d}{\partial \theta_{\text{eff}}} \cdot \frac{\partial \theta_{\text{eff}}}{\partial [\delta(\Delta_{u,k}) + e(\Delta_{u,k})]} \\ &\quad \cdot \frac{\partial e(\Delta_{u,k})}{\partial \Delta_{u,k}} \cdot \frac{\partial \Delta_{u,k}}{\partial \mathbf{w}_k^u[m]}. \end{aligned} \quad (18)$$

But considering the linear approximation model of PCL-PUF where the attacker assumes a constant output for the upper layer and attempts to model the PUF by fully ignoring the upper-layer output, i.e., $\hat{r}_k = c + e(\Delta_{u,k})$ where c is a constant value. The partial derivative of the loss of the linear approximated model is identical to Eq. (18) where the derivatives of both $\delta(\Delta_{u,k})$ and c are equal to zero. Consequently, the partial derivative of the complete PCL-PUF model becomes indistinguishable from that of its linear approximation. From this analysis, we conclude that an LR attack on PCL-PUF effectively reduces to attacking a linear approximation in which the upper-layer weights cannot be optimized and remain fully hidden from the attacker. However, this approximation is inherently weak because it fails to capture the true behavior of the design, as it completely ignores the upper-layer output, which critically influences the lower-layer weights. We further confirm this experimentally in Section V-B1.

3) *Resilience to Reliability Attacks:* To show that PCL-PUF is resilient to reliability attacks through mathematical analysis, we consider the weakest variant, PCL-PUF (1, 1) with $n = 128$, since the resilience of this configuration implies resilience for all stronger configurations with $(K_u, K_d) > (1, 1)$. Our

analysis is not limited to theoretical arguments; we further apply the attack directly against our design and report the results in Section V-B2. The rationale behind the attack is discussed in Section III. To generate noisy CRPs, we follow the same model as explained in Section V-B2.

For the reliability modeling attack to succeed against PCL-PUF, the adversary must recover one layer from the inform signal $\mathcal{I}$, which can be computed as shown in Eq. (4), and use the modeled layer to reconstruct the other, eventually modeling the full design. As discussed in Section III, there is a necessary condition for this attack to succeed on layered PUF designs: the noise of the upper and lower PUFs must have comparable contribution to $\mathcal{I}$. When contributions are unequal, CMA-ES systematically converges to the dominant component whose margin $|\Delta|$ correlates most strongly with $\mathcal{I}$, while failing to recover the weaker component, since $\mathcal{I}$ carries insufficient signal about it. We use this finding to show that the attack is infeasible regardless of which layer the attacker targets first.

We first consider the case where the attacker targets the upper layer. For the upper layer to be recoverable, its noise events must be reflected in $\mathcal{I}$. This requires that when the upper APUF response flips under noise, the final PCL-PUF output also flips. To quantify this effect, we evaluated 90,000 challenges: for each challenge, we computed the upper-layer response and its complement, applied both through the obfuscation mechanism shown in Figure 5, and evaluated the lower layer under both cases. We find that flipping the upper-layer response changes the final output in only $\approx 35\%$ of cases. Consequently, if the upper layer flips with probability β under noise, this propagates to the final output with only $\approx 35\%$ probability, contributing approximately $\beta/3$ to $\mathcal{I}$. By contrast, the lower layer contributes its full noise rate β directly, resulting in an approximately $3\times$ stronger noise contribution than that of the upper layer.

We further confirmed this by measuring the Pearson correlation between each layer's noise-free delay margin Δ and $\mathcal{I}$. Note that we used the true internal weights; these correlations are unavailable to the attacker and represent the theoretical upper bound on what any optimizer can recover from $\mathcal{I}$, computed here only for analysis:

$$\rho_u = \text{corr}(|\Delta_u|, \mathcal{I}), \quad \rho_d = \text{corr}(|\Delta_d^{\theta_{\text{eff}}}|, \mathcal{I}). \quad (19)$$

For PCL-PUF (1, 1) under $\sigma_{\text{noise}} = 0.1 * \sigma_{\text{Process}}$, $E = 11$, and $N = 90,000$ challenges, we obtain $\rho_u = 0.112$ and $\rho_d = 0.302$. The lower layer dominates by a factor of $\approx 2.7\times$, confirming that $\mathcal{I}$ carries insufficient signal for CMA-ES to converge on the upper layer. Following the condition discussed in Section III, the upper layer is thus not recoverable by the reliability attack.

Since the upper layer is unrecoverable, the attacker may instead attempt to target the lower layer first. However, this path is also blocked. The lower-layer response depends on θ_{eff} , which is determined by the upper-layer responses through the obfuscation mechanism shown in Figure 5. Without knowledge of θ_{eff} for each challenge, the attacker cannot construct the correct $\Delta_d^{\theta_{\text{eff}}}$ for the lower layer, making the reliability attack

TABLE I: NN Experiment Settings

Parameter	
Network Architecture (Hidden layers)	[128, 32, 16]
Learning Rate	0.001
Epochs	30
Hidden Layer Activation Function	ReLU
Output Layer Activation Function	Sigmoid
Optimizer	Adam
Loss Function	Binary Cross-Entropy (BCELoss)

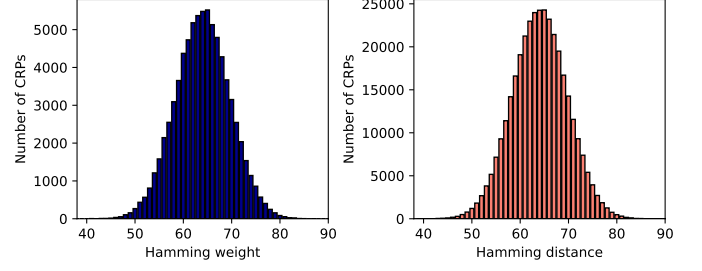


Fig. 6: Statistical distribution of PCL-PUF responses. The left side shows the Hamming weight distribution, while the right side presents the Hamming distance distribution between different instances. Both follow Gaussian distributions, indicating balanced response generation and strong uniqueness.

V. EVALUATION

We implemented a Python-based simulation of PCL-PUF to evaluate its quality and ML resilience. The lower-layer REAP-NVM PUFs were modeled using a 40 nm WO_x ReRAM model [34], with four resistance levels taken directly from the device characterization [34]: 10 k Ω , 75 k Ω , 125 k Ω , and 275 k Ω , and 5% process variation per cell to model PUF behavior. The upper-layer APUFs were simulated using the open-source PyPUF library [35]. Our simulation-based evaluation is in line with the standard practice in PUF research and is widely adopted to assess PUF quality and resilience against modeling attacks [4], [7], [8], [12]–[14], [28].

To assess PUF quality, we simulated 100 PCL-PUF (1,1) instances, collecting 10^6 CRPs per instance. To evaluate resilience against modeling attacks, we compared PCL-PUF with three state-of-the-art designs under various configurations: iPUF [12], XOR PUF [9], and REAP-NVM PUF [13]. Both iPUF and XOR PUF were implemented using the open-source PyPUF library [35]. All evaluated PUF designs were configured with a 128-bit challenge length. Furthermore, we evaluated PCL-PUF under four categories of modeling attacks: LR, following [7]; NNs, following [10] with the architecture specified in Table I; reliability attacks [11] using the open-source DA PUF library [12]; and splitting attacks based on the implementation reported in [8].

A. PUF Quality and Performance

To evaluate the quality of PCL-PUF, we consider two key metrics: uniformity and uniqueness, which measure response bias and inter-device distinctness, respectively. For uniformity, we computed the Hamming weight of PCL-PUF responses across different instances, while for uniqueness we computed the Hamming distance between responses from different instances, as shown in the left and right side of Figure 6. In both cases, the results follow Gaussian distributions, with means of 64, and standard deviations of 5.65 and 5.71, respectively, indicating near-ideal uniformity and uniqueness of approximately 50%.

We further evaluate the obfuscation mechanism described in Section IV-B3 to verify that it does not bias the distribution of the physical modulation bits. Using PCL-PUF (3,1),

where the upper layer consists of three instances, we collected 10,000 randomly selected samples and compared the standard deviation of occurrence frequencies across all 512 modulation-bit configurations before and after obfuscation. The standard deviation remained unchanged after obfuscation (4.42 vs. 4.41), indicating that obfuscation mechanism preserves the original modulation-bit distribution and introduces no observable statistical bias.

B. Resilience Against Modeling Attacks

As defined in the threat model Section IV-A, we evaluate PCL-PUF against four categories of attacks. We first evaluate the resilience of PCL-PUF against LR and reliability attacks to confirm the mathematical analysis presented in Section IV-D2 and Section IV-D3. We then evaluate PCL-PUF against splitting attacks and compare its behavior with iPUF. Finally, since NNs remain the most general applicable modeling attack across all PUF designs, we use them to assess the security of PCL-PUF and compare its resilience with all state-of-the-art PUF designs.

1) *LR Attack*: As our analysis shows in Section IV-D2, LR cannot optimize the full PCL-PUF model; we further confirm this experimentally. We examined this by attacking PCL-PUF (1, 3) with 400,000 CRPs and the mathematical model shown in Eq. (14). The attack achieves a modeling accuracy of approximately 66.63%. In contrast, the same attack with the same number of CRPs achieves 99.98% prediction accuracy on a 4-XOR PUF, which has the same total number of PUF instances as PCL-PUF (1, 3). Moreover, we examined the modeling accuracy of the LR attack against PCL-PUF (1, 5). The maximum achieved accuracy drops to approximately 50%, which is equivalent to random guessing, confirming that increasing K_d further diminishes the effectiveness of the LR attack.

2) *Reliability Attack*: As shown in Section IV-D3, PCL-PUF is resilient against reliability-based attacks because neither the upper nor the lower layer can be modeled independently. We further confirm this experimentally by comparing the attack on PCL-PUF with XOR PUFs.

We conducted 50 independent CMA-ES trials on the upper layer of the PCL-PUF (1, 1), the weakest configuration of

TABLE II: Splitting attack accuracy on iPUF [12] and our PCL-PUF.

Architecture	(K_u, K_d)	Lower-Layer Accuracy (Round 1)	Full Accuracy
iPUF [12]	(3, 3)	91.2%	96%
	(4, 3)	90.6%	96%
PCL-PUF	(3, 3)	66.1%	72.8%
	(4, 3)	64.1%	63.6%

our design. To simulate noisy PUF behavior, additive noise were introduced to each weight, where the noise follows a normal distribution $\mathcal{N}(0, \sigma_{\text{noise}}^2)$, resulting in an effective weight distribution of $\mathcal{N}(0, \sigma_{\text{process}}^2 + \sigma_{\text{noise}}^2)$. To control reliability, we set $\sigma_{\text{noise}} = \gamma \sigma_{\text{process}}$, where $0 \leq \gamma \leq 1$ and $\gamma = 0$ corresponds to a fully reliable PUF. This simulation model is widely adopted in the literature and has been shown to closely match practical behavior [11], [12], [29].

In our experiments, we used $\sigma_{\text{noise}} = 0.1 \cdot \sigma_{\text{process}}$. Each trial used $N = 300,000$ CRPs, $E = 11$ evaluations per challenge, and 30,000 CMA-ES iterations. All trials fail to converge to a meaningful model, with the best achieved accuracy of 51%, consistent with random guessing. In contrast, the same attack successfully recovers a 3-XOR PUF model with 96.4% accuracy using only $N = 40,000$ CRPs.

We further evaluated PCL-PUF (3, 1) to verify that increasing K_u does not alter the dominant contribution of the lower layer to $\mathcal{I}$ and that the upper layer remains unrecoverable. Across all 50 runs, the maximum achievable accuracy across the three upper-layer PUFs is 54.8%, confirming that PCL-PUF remains secure against reliability attacks.

3) *Splitting Attack*: As discussed in Section III, the splitting attack [8] exploits the layered structure of the iPUF by first modeling the lower layer and then recovering the upper layer, ultimately reconstructing the full PUF through a divide-and-conquer strategy. Since PCL-PUF also exhibits a layered architecture, we adapt this attack to our design and evaluate its effectiveness against both PCL-PUF and the iPUF.

We followed the attack procedure of [8] for both architectures, with modifications to accommodate PCL-PUF. To model the lower layer, we approximated the effective modulation bits θ_{eff} by randomly guessing the upper-layer response bits and applying the iterative XOR masking shown in Figure 5 to the input modulation bits θ . After modeling the lower layer, we constructed the upper-layer training set using a heuristic similar to that of [8]. We first discarded challenges for which the upper-layer response did not affect the lower-layer output, as these provide no information about the upper layer. For the remaining sensitive challenges, we enumerated all 2^{K_u} possible combinations of upper layer response bits and evaluated the lower-layer model under each combination. We then randomly selected one combination whose predicted output matches the observed PCL-PUF response. The K_u bits of the selected combination were used as labels for the corresponding APUFs in the upper-layer, producing a labeled training set for the K_u independent upper APUFs, which were modeled separately. The divide-and-conquer refinement was then applied iteratively: the lower layer was retrained using the predicted upper bits,

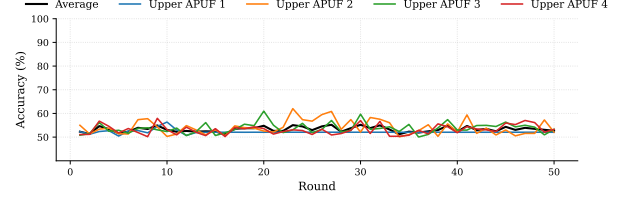


Fig. 7: Upper-layer modeling accuracy over 50 attack rounds for PCL-PUF (4,3). The accuracy of each individual APUF and their average are shown. All accuracies remain close to 50% across all rounds, indicating that the splitting attack fails to recover the upper-layer PUFs and that divide-and-conquer refinement is ineffective against PCL-PUF (4,3).

followed by retraining the upper models, until convergence with limiting the number of iteration to maximum 5 as in the original attack implementation [8].

Table II summarizes the results. The splitting attack achieves high accuracy on iPUF, reaching 96% for both configurations (3, 3) and (4, 3) using 200,000 CRPs. In contrast, the attack is significantly less effective against PCL-PUF, achieving only 72.8% and 63.6% for the same configurations. For the (4, 3) PCL-PUF configuration, the final attack accuracy is less than the accuracy of the lower layer, indicating that modeling the upper layer does not improve the overall model. In this case, the attack fails to extract reliable upper-layer labels and the refinement stage does not converge. These results demonstrate that PCL-PUF provides substantially stronger resilience to the splitting attack than the iPUF at equivalent configurations.

This degradation in attack performance stems from a key architectural difference. In iPUF, the upper-layer output is a single binary bit inserted as an additional digital input to the lower layer. Randomly guessing this bit therefore introduces only input noise, while the underlying lower-layer model remains structurally unchanged and learnable. In contrast, the upper layer of PCL-PUF physically modulates the lower layer rather than providing a digital input. Consequently, incorrect guesses alter the effective physical model of the lower-layer itself, making it significantly harder to isolate and learn. This is reflected in the results, i.e., the lower-layer modeling accuracy of PCL-PUF is consistently lower than that of the iPUF, 66.1% versus 91.2% for the (3,3) configuration.

Furthermore, The upper-layer labeling heuristic used in splitting attacks also degrades more severely for PCL-PUF. In the iPUF, the attacker evaluates only two possibilities for the single interpose bit and selects the one that produces the correct response. In contrast, in PCL-PUF the upper layer produces K_u bits, requiring the attacker to evaluate all 2^{K_u} possible combinations. On average, half of these combinations yield a correct response, yet only one corresponds to the true upper-layer output. The attacker must therefore randomly select one of the $2^{K_u}/2$ valid combinations, introducing significant label noise into the upper-layer training set.

This effect is evident in the PCL-PUF (4, 3) configuration, where the attacker must choose among an average of eight candidate combinations. Consequently, the probability

of selecting the correct upper-layer response across all PUFs simultaneously is only $1/8 = 0.125$, making reliable recovery extremely difficult. This is reflected in the results, where the first-round lower-layer accuracy is identical to the final PCL-PUF attack accuracy, indicating that the divide-and-conquer refinement fails to improve the model. Increasing K_u further strengthens the security of the upper layer, as the number of candidate combinations grows exponentially, rapidly reducing the probability of correctly recovering the true upper-layer responses.

To confirm that PCL-PUF (4, 3) fully mitigates the splitting attack, we performed 50 divide-and-conquer refinement rounds, which is 10x more than typically used in prior work, and recorded the accuracy of each upper-layer APUF across rounds. As shown in Figure 7, all four upper-layer APUFs remain near 50% accuracy throughout the rounds, confirming that divide-and-conquer refinement is effectively disabled. Note that upper-layer accuracy is not observable to the attacker and is reported here for analysis only.

4) *Neural Network Attack*: PCL-PUF demonstrates strong resilience against state-of-the-art attacks, including LR, reliability, and splitting attacks. Since NN attacks are applicable to all PUF designs, as they operate as black-box optimizers, we adopt them as a benchmark for evaluating the security margin of PCL-PUF and comparing it with state-of-the-art designs, including REAP-NVM PUF [13], different variants of iPUF [12], and XOR PUF [9].

For this evaluation, we generated a dataset of 10^7 CRPs, using 80% of the data for training and 20% for testing. To observe the effect of increasing CRPs on attack performance, we performed the attack incrementally, starting with 10^3 CRPs and increasing by one order of magnitude at each step up to 10^7 CRPs. We chose 10^7 as the upper bound because, in NVM-based PUFs, collecting additional CRPs beyond this point may exceed practical endurance limits [33], causing the PUF to fail before an attacker can obtain a useful model. We consider a PUF to be *broken* if the modeling accuracy exceeds 90%.

Figure 8 summarizes the modeling accuracy of various PUF designs against the NN architecture detailed in Table I. The left side of the figure shows the modeling accuracy of state-of-the-art PUFs across different configurations, while the right side presents the results for various PCL-PUF configurations alongside the best-performing configuration of state-of-the-art PUFs.

REAP-NVM PUF exhibits resilience against NN modeling attacks for up to 10^4 CRPs; however, modeling accuracy rises beyond this point linearly, reaching approximately 88% and 98.8% at 10^6 and 10^7 CRPs respectively. In contrast, iPUF configurations show varying resilience depending on the configuration: the iPUF (1, 1) can be fully broken at 10^5 CRPs, while the (3, 1) and (3, 3) configurations require more CRPs to be broken, achieving roughly 97% modeling accuracy at 10^6 CRPs. The iPUF (1, 5) and (3, 5) configurations represent the best candidates for resilience against modeling attacks: (1, 5) is fully broken at 10^7 CRPs achieving 98% modeling accuracy, whereas (3, 5) can only be modeled with approximately 74%

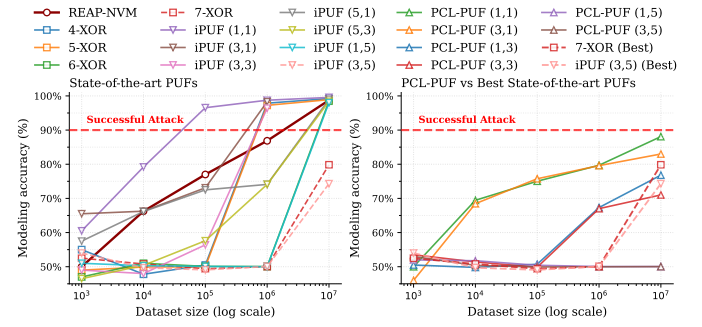


Fig. 8: Modeling accuracy of PCL-PUF and state-of-the-art PUFs under NN attack across different CRP dataset sizes. The left side compares state-of-the-art PUFs across different configurations (e.g., varying the number of APUFs in the upper and lower layers of iPUF designs). The right side presents different PCL-PUF configurations evaluated against the strongest state-of-the-art configuration.

modeling accuracy. XOR PUF configurations also exhibit varying resilience: a 6-XOR PUF is fully broken at 10^7 CRPs with 98.5% accuracy, whereas a 7-XOR PUF reaches a maximum modeling accuracy of approximately 80%. In summary, all state-of-the-art configurations can be broken with 10^7 CRPs, except for the 7-XOR PUF and the iPUF (3, 5).

In contrast, **none** of the PCL-PUF configurations can be fully broken by NN attacks, including the weakest variants (1, 1) and (3, 1), where the modeling accuracy reaches only 88% and 83%, respectively. This demonstrates significantly stronger resilience to NN modeling attacks compared to state-of-the-art PUFs. Even with 10^7 which approaches the practical endurance limit for NVM-based PUFs, the maximum achievable modeling accuracies for the PCL-PUF (3, 3) and PCL-PUF (1, 3) are limited to 71% and 77%, respectively, which is comparable to the best configuration of the state-of-the-art candidates: the 7-XOR PUF and iPUF (3, 5), achieving 80% and 74%, respectively, despite requiring more PUF instances. The best-performing PCL-PUF configurations, (1, 5) and (3, 5), remain close to random guessing, achieving approximately 50% modeling accuracy.

These results indicate that, in general, PCL-PUF achieves very high resilience against NN modeling attacks even with a smaller number of PUF instances, as shown by the (3, 3) and (1, 3) configurations. Moreover, stronger configurations such as (1, 5) and (3, 5) remain fully resilient to NN attacks up to 10^7 CRPs, with modeling accuracy limited to random guessing.

C. Comparison Against State-of-the-Art PUFs

After evaluating PCL-PUF against LR, reliability, splitting, and NN attacks, Table III compares the overall security of PCL-PUF against state-of-the-art designs. Prior works, such as XOR PUF, offer good resilience against NN attacks, but remain vulnerable to reliability attacks [11] and LR attacks [7]. iPUF mitigates these vulnerabilities, providing strong resilience against LR and reliability attacks; however, it is susceptible to splitting attacks [8]. Our PCL-PUF simultaneously mitigates all these attack vectors while offering stronger resilience against NN modeling attacks compared to the state of the art.

TABLE III: Security comparison of Our PCL-PUF against strongest configuration of state-of-the-art PUF designs.

PUF	LR [7]	Reliability [11]	Splitting [8]	NN @ 10^7 CRPs
XOR PUF [9]	✓	✓	—	$\approx 80\%$ (7-XOR)
iPUF [12]	×	×	✓	$\approx 74\%$ (3,5)
PCL-PUF	×	×	×	$\approx 50\%$ (3,5)

✓ = broken, × = resilient, — = not applicable

VI. DISCUSSION

The weakest PCL-PUF configurations, (1,1) and (3,1), mitigate LR and reliability-based attacks but reach modeling accuracies of 88% and 83% under NN attacks with 10^7 CRPs, leaving only a small margin below the 90% threshold. For stronger resilience against both splitting and NN attacks, configurations with $K_d \geq 3$ and $K_u \geq 3$ are recommended. While (1,5) and (3,5) both limit NN modeling accuracy to a random guessing at 10^7 CRPs, (3,5) offers stronger resilience to splitting attacks by exponentially increasing the complexity of recovering the upper-layer model compared to (1,5).

Using only the lower layer of the PCL-PUF, without the upper layer, yields a design similar to K_d -XOR REAP-NVM PUFs. This structure can strongly mitigate NN attacks, as NN behaves like a black-box optimizer, but remains vulnerable to reliability and LR attacks [7], [11], [12], [28]. Increasing K_d improves resilience to LR attacks [7], [8] but does not prevent reliability attacks [11], which reduce the complexity from sub-exponential to linear in K_d [11]. Thus, the lower layer alone is insufficient for security.

The upper layer of the PCL-PUF is essential. It counters reliability attacks by introducing unequal noise contributions to the output, as discussed in Section IV-D3, and thwarts LR attacks by creating structural dependence between the layers that causes gradients to vanish during training, as discussed in Section IV-D2. Hence, only the complete PCL-PUF, with both upper and lower layers, offers strong simultaneous resilience against all state-of-the-art attacks, including LR, reliability, splitting, and NN attacks.

Simulation-based evaluation is standard of the practice in PUF research and is widely adopted for assessing both quality metrics and ML resilience even in design when silicon prototypes on FPGA are available [4], [7], [8], [12], [28]. Simulation is particularly well-suited for NVM-based designs, which cannot be prototyped on FPGAs and are costly to fabricate [13], [14]. Furthermore, commercial NVM chips do not expose MLC programming interfaces, making physical evaluation of MLC-dependent PUF designs infeasible without custom silicon, leaving simulation as the only practical evaluation methodology for this class of designs [13]–[15].

VII. CONCLUSION

In this work, we presented PCL-PUF, a strong PUF architecture designed to mitigate state-of-the-art modeling attacks. It physically couples delay-based and NVM-based PUFs via challenge-dependent resistance modulation. This coupling produces dynamically varying challenge–response behavior, prevents independent learning of internal components, and

significantly increases the complexity of modeling attacks. We showed both mathematically and experimentally that PCL-PUF exhibits resilience against LR and reliability attacks. Compared to iPUF [12], PCL-PUF offers much stronger resilience to splitting attacks, with 63.6% modeling accuracy versus 96% for iPUF. Under NN attacks with up to 10^7 CRPs, PCL-PUF remains near 50% modeling accuracy, while state-of-the-art designs of similar hardware size reach more than 98%. Unlike prior PUFs, which remain vulnerable to at least one modeling attack, PCL-PUF simultaneously resists all evaluated attack classes, making it a strong candidate for lightweight hardware security in resource-constrained systems.

REFERENCES

- [1] R. Pappu, B. Recht, J. Taylor, and N. Gershenfeld, “Physical one-way functions,” *Science*, vol. 297, no. 5589, pp. 2026–2030, 2002.
- [2] J. Delvaux, R. Peeters, D. Gu, and I. Verbauwhede, “A survey on lightweight entity authentication with strong pufs,” *ACM Computing Surveys (CSUR)*, vol. 48, no. 2, pp. 1–42, 2015.
- [3] M. Alsharkawy, H. Nassar, J. González-Gómez, X. Xiao, O. Abboud, and J. Henkel, “Dpref: Decentralized key generation using physical-related functions,” *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 5s, pp. 1–24, 2025.
- [4] G. T. Becker, “On the pitfalls of using arbiter-pufs as building blocks,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 8, pp. 1295–1307, 2015.
- [5] C. Yehoshuva, R. Raja Adhithan, and N. Nalla Anandakumar, “A survey of security attacks on silicon based weak puf architectures,” in *International Symposium on Security in Computing and Communication*. Springer, 2020, pp. 107–122.
- [6] H. Nassar, L. Bauer, and J. Henkel, “Effects of runtime reconfiguration on pufs implemented as fpga-based accelerators,” *IEEE Embedded Systems Letters*, vol. 15, no. 4, pp. 174–177, 2023.
- [7] U. Rührmair, F. Sehnke, J. Sölter, G. Dror, S. Devadas, and J. Schmidhuber, “Modeling attacks on physical unclonable functions,” in *Proceedings of the 17th ACM conference on Computer and communications security*, 2010, pp. 237–249.
- [8] N. Wisiol, C. Mühl, N. Pirnay, P. H. Nguyen, M. Margraf, J.-P. Seifert, M. Van Dijk, and U. Rührmair, “Splitting the interpose puf: A novel modeling attack strategy,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 97–120, 2020.
- [9] G. E. Suh and S. Devadas, “Physical unclonable functions for device authentication and secret key generation,” in *Proceedings of the 44th annual design automation conference*, 2007, pp. 9–14.
- [10] P. Santikellur, A. Bhattacharyay, and R. S. Chakraborty, “Deep learning based model building attacks on arbiter puf compositions,” *Cryptology ePrint Archive*, 2019.
- [11] G. T. Becker, “The gap between promise and reality: On the insecurity of xor arbiter pufs,” in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2015, pp. 535–555.
- [12] P. H. Nguyen, D. P. Sahoo, C. Jin, K. Mahmood, U. Rührmair, and M. Van Dijk, “The interpose puf: Secure puf design against state-of-the-art machine learning attacks,” *Cryptology ePrint Archive*, 2018.
- [13] H. Nassar, M.-L. Wei, C.-L. Yang, J. Henkel, and K.-H. Chen, “Reap-nvm: Resilient endurance-aware nvm-based puf against learning-based attacks,” in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–2, see also arXiv version: <https://arxiv.org/abs/2501.06367>.
- [14] H. Nassar, L. Bauer, and J. Henkel, “Anv-puf: Machine-learning-resilient nvm-based arbiter puf,” *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 5s, pp. 1–23, 2023.
- [15] R. Ali, H. Ma, Z. Hou, D. Zhang, E. Deng, and Y. Wang, “A reconfigurable arbiter mpuf with high resistance against machine learning attack,” *IEEE Transactions on Magnetics*, vol. 57, no. 10, pp. 1–7, 2021.
- [16] C.-H. Chang, Y. Zheng, and L. Zhang, “A retrospective and a look forward: Fifteen years of physical unclonable function advancement,” *IEEE Circuits and Systems Magazine*, vol. 17, no. 3, pp. 32–62, 2017.
- [17] R. Pappu, B. Recht, J. Taylor, and N. Gershenfeld, “Physical one-way functions,” *Science*, vol. 297, no. 5589, pp. 2026–2030, 2002.

- [18] A. Maiti, V. Gunreddy, and P. Schaumont, "A systematic method to evaluate and compare the performance of physical unclonable functions," in *Embedded systems design with FPGAs*. Springer, 2012, pp. 245–267.
- [19] J. Zhang, C. Shen, Z. Guo, Q. Wu, and W. Chang, "Ct puf: Configurable tristate puf against machine learning attacks for iot security," *IEEE Internet of Things Journal*, vol. 9, no. 16, pp. 14 452–14 462, 2021.
- [20] A. Chen, "A review of emerging non-volatile memory (nvm) technologies and applications," *Solid-State Electronics*, vol. 125, pp. 25–38, 2016.
- [21] J. Henkel, L. Siddhu, H. Nassar, L. Bauer, J.-J. Chen, C. Hakert, T. Seidl, K. H. Chen, X. S. Hu, M. Li *et al.*, "Co-designing nvm-based systems for machine learning and in-memory search applications," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024, pp. 1–8.
- [22] N. Noor and H. Silva, "Phase change memory for physical unclonable functions," in *Applications of Emerging Memory Technology: Beyond Storage*. Springer, 2019, pp. 59–91.
- [23] C. Pan, M. Xie, C. Yang, Y. Chen, and J. Hu, "Exploiting multiple write modes of nonvolatile main memory in embedded systems," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 16, no. 4, pp. 1–26, 2017.
- [24] F. Zahoor, T. Z. Azni Zulkifli, and F. A. Khanday, "Resistive random access memory (rram): An overview of materials, switching mechanism, performance, multilevel cell (mlc) storage, modeling, and applications," *Nanoscale research letters*, vol. 15, no. 1, p. 90, 2020.
- [25] K. Kursawe, A.-R. Sadeghi, D. Schellekens, B. Skoric, and P. Tuyls, "Reconfigurable physical unclonable functions-enabling technology for tamper-resistant storage," in *2009 IEEE International Workshop on Hardware-Oriented Security and Trust*. IEEE, 2009, pp. 22–29.
- [26] H. Nassar, M.-L. Wei, C.-L. Yang, J. Henkel, and K.-H. Chen, "Reap-nvm: Resilient endurance-aware nvm-based puf against learning-based attacks," in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–2.
- [27] Y. Gao, D. C. Ranasinghe, S. F. Al-Sarawi, O. Kavehei, and D. Abbott, "Emerging physical unclonable functions with nanotechnology," *IEEE access*, vol. 4, pp. 61–80, 2017.
- [28] J. Tobisch and G. T. Becker, "On the scaling of machine learning attacks on pufs with application to noise bifurcation," in *International Workshop on Radio Frequency Identification: Security and Privacy Issues*. Springer, 2015, pp. 17–31.
- [29] J. Delvaux and I. Verbauwhede, "Side channel modeling attacks on 65nm arbiter pufs exploiting cmos device noise," in *2013 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)*. IEEE, 2013, pp. 137–142.
- [30] Y. Hori, H. Kang, T. Katashita, and A. Satoh, "Pseudo-lfsr puf: A compact, efficient and reliable physical unclonable function," in *2011 International Conference on Reconfigurable Computing and FPGAs*. IEEE, 2011, pp. 223–228.
- [31] H. Nassar, L. Bauer, and J. Henkel, "Capuf: Cascaded puf structure for machine learning resiliency," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4349–4360, 2022.
- [32] M. Alsharkawy, J. Zwerschke, H. Nassar, J. González-Gómez, and J. Henkel, "Late breaking results: The hidden risks of activation duration in plpufs," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2025, pp. 1–2.
- [33] G. W. Burr, M. J. Breitwisch, M. Franceschini, D. Garetto, K. Gopalakrishnan, B. Jackson, B. Kurdi, C. Lam, L. A. Lastras, A. Padilla *et al.*, "Phase change memory technology," *Journal of Vacuum Science & Technology B*, vol. 28, no. 2, pp. 223–262, 2010.
- [34] W.-C. Chien, M.-H. Lee, F.-M. Lee, Y.-Y. Lin, H.-L. Lung, K.-Y. Hsieh, and C.-Y. Lu, "Multi-level 40nm wo x resistive memory with excellent reliability," in *2011 International electron devices meeting*. IEEE, 2011, pp. 31–5.
- [35] N. Wisiol, C. Gräbnitz, C. Mühl, B. Zengin, T. Soroceanu, N. Pirnay, K. T. Mursi, and A. Baliuka, "pypuf: Cryptanalysis of Physically Unclonable Functions," 2021. [Online]. Available: <https://doi.org/10.5281/zenodo.3901410>